



Submitted By:

Abdullah Saeed – 2025-CYS-51

Assignment:

Project No. 1

Submitted To:

Sir Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer Engineering

Create a Pacman game using OOP that involves the use of multiple classes and objects that interact with each other. The PackmanGame class contains the main method and initializes the game by creating instances of the GameBoard, Pacman, Ghost, and Food classes. The PacmanGame class also contains the game loop and handles user input, allowing the player to control Pacman's movement around the board.

```
package Project1;

import java.util.Scanner;
import java.util.Random;
import java.util.*;

class GameBoard {
    private int size = 10;
    private char[][] maze = {
        {'#','#','#','#','#','#','#','#','#','#'},
        {'#','.','.','.','.','.','.','.','.','#'},
        {'#','.','#','#','.','#','#','.','.','#'},
        {'#','.','.','#','.','.','#','.','.','#'},
        {'#','#','.','#','#','.','#','#','.','#'},
        {'#','.','.','.','.','.','.','.','#','.','#'},
    }
```

```

        {'#','.', '#','#','#','#','.', '#','.', '#'},
        {'#','.', '.', '.', '.', '#','.', '.', '.', '#'},
        {'#','.', '.', '#','.', '.', '.', '#','.', '#'},
        {'#','#','#','#','#','#','#','#','#','#'}
    };

    public void display(Pacman p, Ghost g) {
        for (int i=0;i<size;i++) {
            for (int j=0;j<size;j++) {
                if (p.getX()==i && p.getY()==j) {
                    System.out.print("P ");
                }
                else if (g.getX()==i && g.getY()==j) {
                    System.out.print("G ");
                }
                else {
                    System.out.print(maze[i][j] + " ");
                }
            }
            System.out.println();
        }
    }

    public boolean isWall(int x, int y) {
        return maze[x][y]=='#';
    }

    public void checkFood(Pacman p) {
        if (maze[p.getX()][p.getY()]=='.') {
            maze[p.getX()][p.getY()=' '];
            p.increaseScore();
        }
    }

    public int getSize() {

```

```

        return size;
    }
}

class Pacman {
    private int x, y;
    private int score;
    public Pacman(int x, int y) {
        this.x=x;
        this.y=y;
        score=0;
    }
    public void move(int dx, int dy) {
        x +=dx;
        y +=dy;
    }
    public void increaseScore() {
        score++;
    }
    public int getScore() {
        return score;
    }
    public int getX() {return x; }
    public int getY() {return y; }
}

class Ghost {
    private int x, y;
    private Random rand=new Random();
    public Ghost(int x, int y) {

```

```

        this.x = x;
        this.y = y;
    }
    public void randomMove() {
        int dx=rand.nextInt(3)-1;
        int dy=rand.nextInt(3)-1;
        x += dx;
        y += dy;
    }
    public void setPosition(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public int getX() {return x; }
    public int getY() {return y; }
}
class Food {
    private int x, y;
    private boolean eaten;
    public Food(int x,int y) {
        this.x=x;
        this.y=y;
        eaten=false;
    }
    public void eat() {
        eaten=true;
    }
    public boolean isEaten() {

```

```

        return eaten;
    }}
public class PacmanGame {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        GameBoard board=new GameBoard();
        Pacman pacman=new Pacman(1, 1);
        Ghost ghost=new Ghost(8, 8);
        boolean running=true;
        while (running) {
            System.out.println("\nScore: " + pacman.getScore());
            board.display(pacman, ghost);
            System.out.print("Move (W/A/S/D): ");
            char move=sc.next().toUpperCase().charAt(0);
            int dx=0,dy=0;
            switch (move) {
                case 'W':dx=-1; break;
                case 'S':dx=1; break;
                case 'A':dy=-1; break;
                case 'D':dy=1; break;
                default: System.out.println("Invalid Input");
            }
            int newX=pacman.getX() + dx;
            int newY=pacman.getY() + dy;
            if (newX >= 0 && newX < board.getSize() &&
                newY >= 0 && newY < board.getSize() &&
                !board.isWall(newX, newY)) {
                pacman.move(dx, dy);
            }
        }
    }
}

```

```

    }
    int oldX=ghost.getX();
    int oldY=ghost.getY();
    ghost.randomMove();
    if (ghost.getX() < 0 ||ghost.getX()>=board.getSize() ||
        ghost.getY()<0 ||ghost.getY()>=board.getSize() ||
        board.isWall(ghost.getX(), ghost.getY())) {
        ghost.setPosition(oldX, oldY);}
    board.checkFood(pacman);
    if (pacman.getX()==ghost.getX() &&
        pacman.getY()==ghost.getY()) {
        System.out.println("Game Over!");
        running=false;
    }
    System.out.println("Final Score: " + pacman.getScore());
    sc.close();
}

```

OUTPUT:

```
Run  PacmanGame x
# # # # # # # #
# . #
# . # # . # # . #
# . . # . . # . #
# # . # # . # # #
# . . . . . # #
# . # # # # . # P #
# . . . . # . . G #
# . . # . . . # . #
# # # # # # # # #
Move (W/A/S/D): s
Game Over!
Final Score: 13

Process finished with exit code 0
```

PROJECT REPORT



University of Engineering and Technology, Lahore
Department of Computer Engineering

Course: Object Oriented Programming (CMPE-221L)

Lab Title: Pac-Man Game

Assignment: Complex Engineering Problem (CEP)

Instructor: Mr. Shahmeer Nawaz

Semester: 2nd (Session 2025)

Submitted By:

Abdullah Saeed

Roll No: 2025-CYS-51

Submission Date: 18-APR-26

Introduction

Pac-Man is a classic arcade game in which the player controls a character that navigates through a maze while collecting food and avoiding enemies.

This project implements a simplified Pac-Man game using **Object-Oriented Programming (OOP) in Java**, where multiple classes interact with each other to simulate real-time gameplay.

Objectives

- To implement a game using OOP principles
 - To design interacting classes and objects
 - To understand game logic and control flow
 - To implement collision detection and scoring
-
-

Complex Engineering Problem (CEP)

– Depth of Knowledge

The project demonstrates:

- Encapsulation
 - Aggregation
 - Composition
 - Class-based design
-

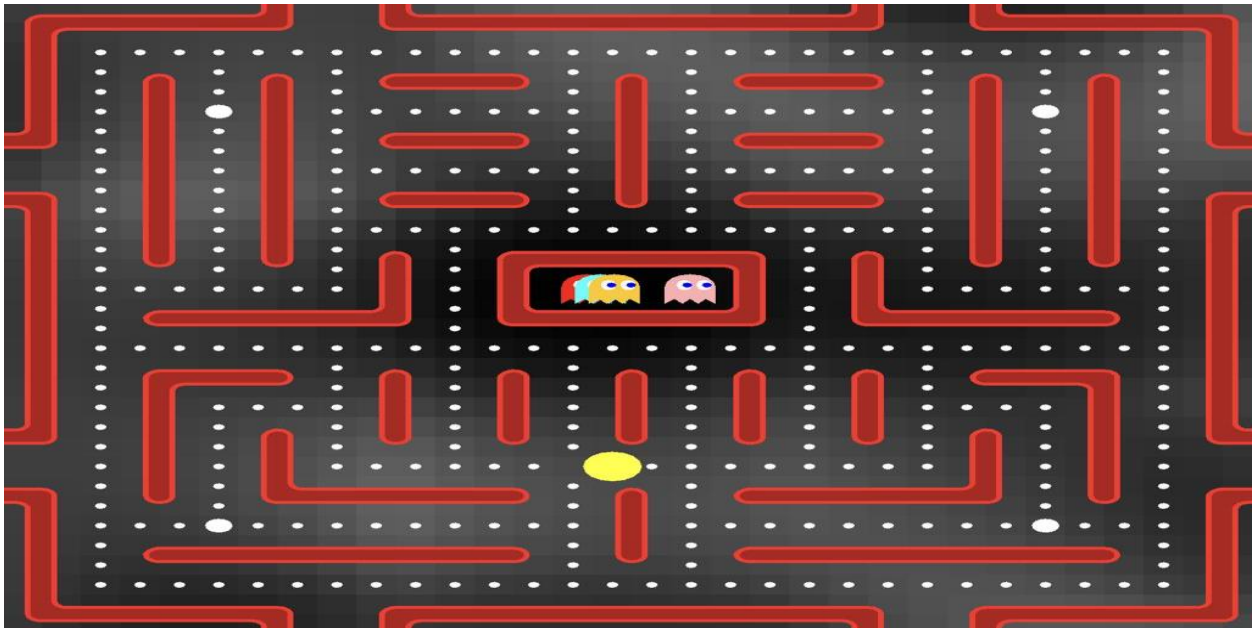
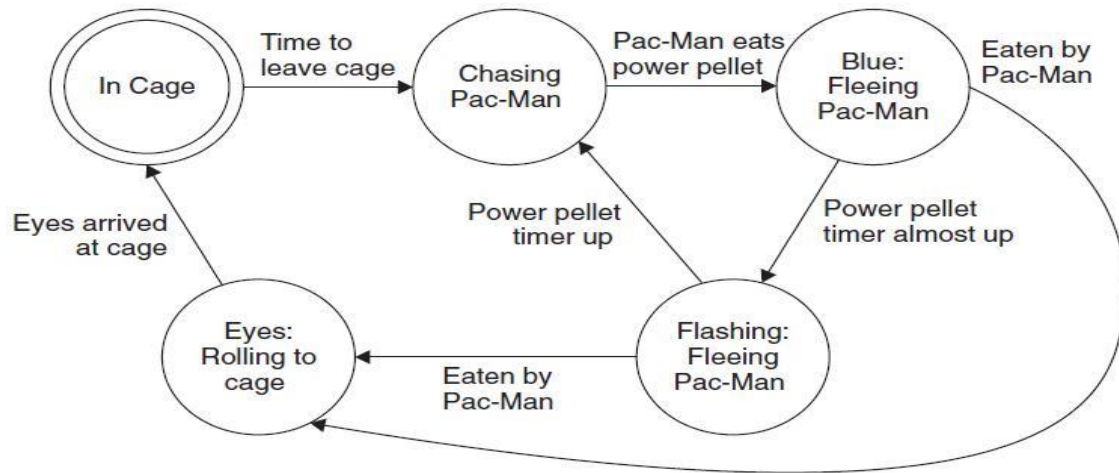
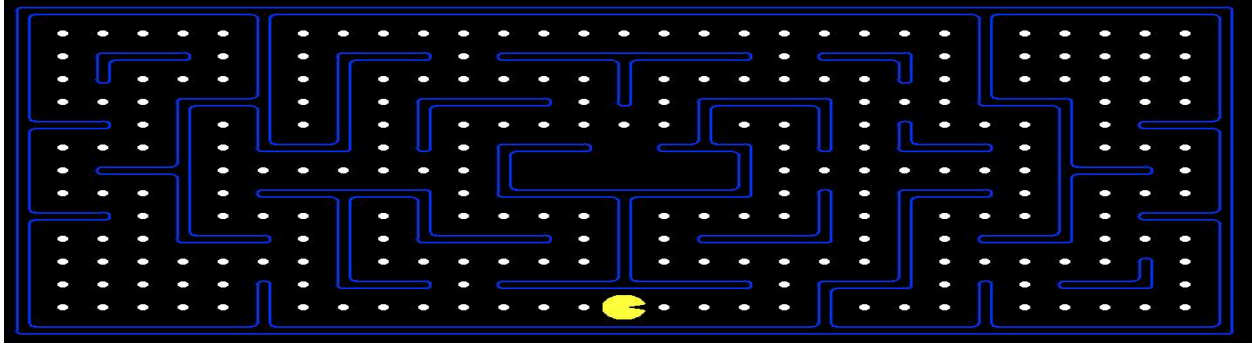
– Range of Conflicting Requirements

- Game complexity vs user simplicity
 - Realism vs playability
 - Difficulty vs user satisfaction
-

– Depth of Analysis

- Game mechanics and movement
 - Maze (2D array) representation
 - Input handling using Scanner
 - Testing and debugging
-

System Architecture



Components:

- **PacmanGame** Controls game loop and input
 - **GameBoard** Maze and food system
 - **Pacman** Player
 - **Ghost** Enemy
 - **Food** Food representation
-

Class Design

PacmanGame

- Main class
- Initializes objects
- Controls game loop

GameBoard

- Stores maze using 2D array
- Handles walls and food

Pacman

- Stores position and score
- Handles movement

Ghost

- Moves randomly
- Acts as enemy

Food

- Represents food items in game
-

OOP Concepts Used

Encapsulation

Data (position, score) is stored inside classes using private variables.

Aggregation

GameBoard contains maze and food elements.

Composition

PacmanGame creates and manages all objects.

Abstraction

Each class performs a specific role.

Working of the Game

1. Game starts
 2. Maze is displayed
 3. User inputs movement (W/A/S/D)
 4. Pac-Man moves in maze
 5. Ghost moves randomly
 6. If Pac-Man eats food score increases
 7. If Pac-Man hits ghost game over
-

Output:

```
Run  PacmanGame x
Move (W/A/S/D): a

Score: 21
# # # # # # # # # #
# . # # # # # # #
# . # # . # # . #
# . . # . . # . #
# # . # # . # # #
# . P # # # # #
# G # # # # # #
# . . . . # #
# . . # . . . # . #
# # # # # # # # # #
Move (W/A/S/D): a
Game Over!
Final Score: 22
```

Features

- Maze with walls
- Player control
- Enemy movement
- Collision detection
- Score system

Limitations

- Console-based (no GUI)
- Basic ghost logic
- Single level

Future Improvements

- GUI using JavaFX
- Multiple levels
- Smarter ghost AI
- Sound effects

Conclusion

This project successfully demonstrates the use of Object-Oriented Programming in building a Pac-Man game. It shows how multiple classes interact to form a complete system and helps in understanding real-world software design.
